



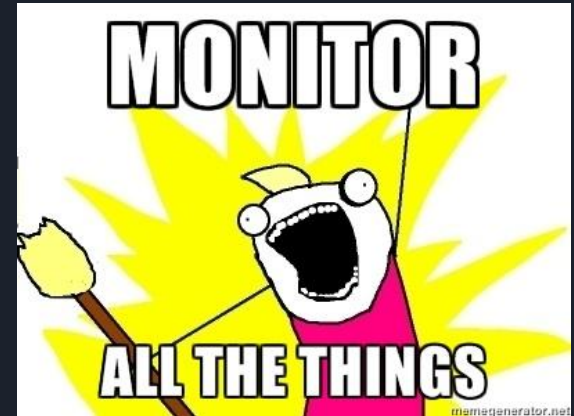
# Introduction to Monitoring

COMS W4156  
November 19, 2024

Prepared by: Bin Choi (bc3159) &  
Shivansh Srivastava (ss5945)

# What is Monitoring?

- Focusing on tracking predefined metrics and key performance indicators (KPIs) to detect deviations from expected behavior
- Necessary for ensuring system reliability, availability, and performance
- Types of monitoring:
  - Infrastructure (e.g., servers, networks, storage)
  - Application (e.g., APIs, databases, backend systems)
  - UX (e.g., latency, errors, usability)
  - Security (e.g., threat detection, vulnerabilities)





# Frameworks & Tools

- Key tools used in the industry:
  - Open-source: Grafana, Prometheus, ELK Stack
  - Enterprise: Datadog, Splunk, New Relic, AppDynamics
- Features:
  - Visualization capabilities
  - Real-time monitoring
  - Scalability and integrations with cloud platforms



# What is a Dashboard?

- Used for visualizing data: Metrics, logs, alerts
- Design principles for effective dashboards:
  - Clear and actionable visualizations
  - Avoiding information overload
  - Using thresholds, color coding, and alerts
- Examples of dashboards:
  - System health metrics (e.g., CPU, memory)
  - Application performance metrics (e.g., API response times)
  - Business KPIs (e.g., revenue per transaction)



New dashboard ▾



Zoom Out



an hour ago to a few seconds ago ▾



Dashboards



Data Sources



admin



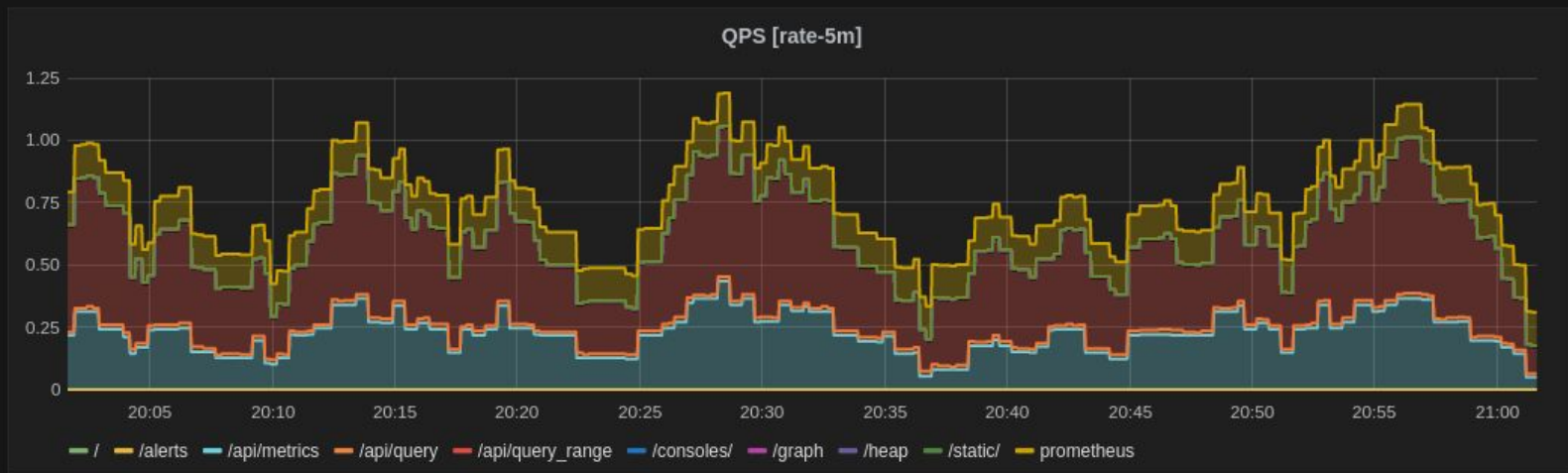
Main Org. ▾



Grafana admin



Sign out



Graph

General

Metrics

Axes & Grid

Display Styles

Time range

[Back to dashboard](#)

|  |               |                                                                      |        |             |  |  |
|--|---------------|----------------------------------------------------------------------|--------|-------------|--|--|
|  | Query         | rate(http_request_duration_microseconds_count{job="prometheus"}[5m]) | Metric | metric name |  |  |
|  | Legend format | {{handler}}                                                          |        |             |  |  |
|  | Max points    | auto                                                                 |        |             |  |  |



demo-prometheus ▾

+ Add query



# Anomaly Detection

- Understanding anomalies:
  - Point
  - Contextual
  - Collective
- Techniques for anomaly detection:
  - Simple thresholds - Fixed value alerts
  - Statistical methods - Moving averages, standard deviations, z-scores
  - ML approaches - Clustering, outlier detection, predictive modeling
- Real-world examples:
  - Detecting spikes in HTTP error rates
  - Identifying memory leaks over time



# Logging & Tracing

- Aggregating logs across multiple systems for a unified view
  - Tools:
    - Fluentd
    - Logstash
    - ELK Stack
- Distributed tracing for debugging: Capturing end-to-end flow of a request across services
  - Tools:
    - Jaeger
    - Zipkin
- Ex.
  - Tracing a failed request in a microservices architecture

# Alerting Mechanisms

- Setting up alerts:
  - Threshold-based alerts
  - Behavior-based alerts using anomaly detection
- Types of alerts:
  - Real-time alerts (e.g., email, SMS, Slack)
  - Incident response tools: PagerDuty, Opsgenie
- Best practices:
  - Reducing false positives
  - Defining priority levels for alerts





# Troubleshooting & Root Cause Analysis

- Workflow for resolving issues:
  - Detect: Identifying the anomaly or issue
  - Investigate: Using logs, metrics, and tracing to pinpoint the cause
  - Resolve: Implementing fixes or applying temporary patches
  - Prevent: Conducting post-mortem analyses and defining preventive measures
- Ex.
  - Debugging high latency in an API
  - Resolving server resource exhaustion



# Performance Monitoring

| Term                    | Description                                                    | Example                                                         |
|-------------------------|----------------------------------------------------------------|-----------------------------------------------------------------|
| Service Level Agreement | Formal agreement with consequences for failing to meet the SLO | 10% credit if uptime < 99.9%;<br>25% credit if uptime < 99.0%   |
| Service Level Objective | Sets the target for acceptable service performance             | 99.9% of API requests must meet the SLI within the last 30 days |
| Service Level Indicator | Evaluates whether SLO is met                                   | Percentage of successful API requests within 100ms              |



# Demo Prework: Prometheus

- An open-source system monitoring and alerting toolkit designed for reliability and scalability
- Core features:
  - Time-series data collection with a powerful query language (PromQL)
  - Multi-dimensional data model using key-value pairs
  - Pull-based metric collection through HTTP endpoints
  - Alerting capabilities via integrations like Alertmanager
- Use cases:
  - Infrastructure monitoring
  - Application performance tracking
  - Custom metrics





# Demo Prework: Prometheus Metric Types

- Prometheus organizes data into four metric types, each suited for specific use cases:
  - Counter
    - A cumulative metric that only increases
    - Ex. Total HTTP requests processed (`http_requests_total`)
  - Gauge
    - Represents a value that can increase or decrease
    - Ex. Current memory usage (`memory_usage_bytes`)
  - Histogram
    - Measures distributions, dividing data into configurable buckets
    - Ex. HTTP request latency (`http_request_duration_seconds`)
  - Summary
    - Similar to Histogram but includes quantiles (e.g. 95th percentile)
    - Ex. Request durations broken into percentiles



# Demo Prework: PromQL

- A query language designed specifically for Prometheus to extract and manipulate time-series data
- Core features:
  - Supports filtering and aggregation based on labels (key-value pairs)
  - Allows mathematical operations on metrics
- Ex.
  - Calculate Request Rate:
    - `rate(http_requests_total[5m])` - Calculates the average request rate over the last 5 min
  - Monitor CPU Usage:
    - `avg by (instance) (rate(cpu_usage_seconds_total[5m]))` - Averages CPU usage over the last 5 min grouped by instance
  - Error Rate:
    - `sum(rate(http_requests_total{status="500"}[5m]))` - Total rate of HTTP 500 errors over 5 min



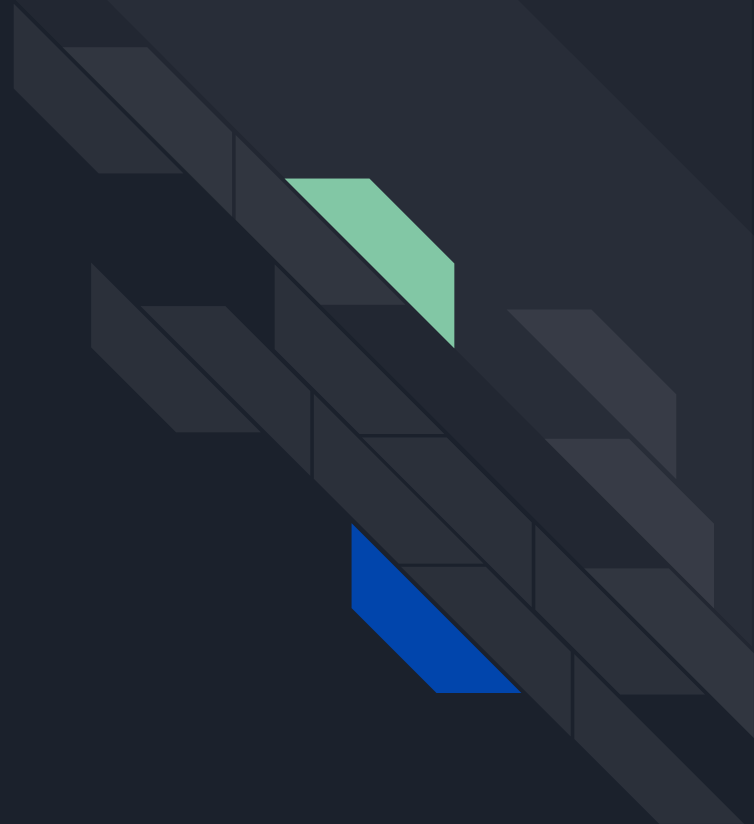
# Demo Prework: Grafana

- An open-source visualization and analytics platform for monitoring and observability
- Core features:
  - Supports multiple data sources (e.g. Prometheus, Elasticsearch, InfluxDB)
  - Rich visualization options: graphs, heatmaps, tables
  - Configurable alerts with threshold-based notifications
  - Dashboards tailored for various stakeholders (e.g. DevOps, business)
- Use cases:
  - Real-time monitoring dashboards
  - Alert management
  - Business analytics



Grafana

Demo



Questions?

